

Tentu, saya akan menjelaskan kode `DragDropManager.cs` ini secara mendetail, membedahnya bagian per bagian, dan menjelaskan alur logika serta fungsi dari setiap baris kodenya.

Secara umum, kelas ini berfungsi sebagai **pengelola interaksi *drag-and-drop* (seret dan lepas)** di dalam aplikasi MiniPhotoshop. Ia mengatur perpindahan gambar antara Canvas utama (tempat edit) dan daftar Thumbnail (tempat penyimpanan sementara).

Berikut adalah penjelasan rincinya:

1. Library Import (Using Statements)

C#

```
using MiniPhotoshop.Logic;  
using System.Drawing;  
using System.Windows.Forms;  
using System.Linq; // <-- PERBAIKAN 1  
using System.Collections.Generic; // <-- Diperlukan untuk List<>
```

- **MiniPhotoshop.Logic:** Mengimpor logika bisnis aplikasi (seperti `ImageEditorService`).
- **System.Drawing:** Diperlukan untuk memanipulasi objek grafis, khususnya kelas `Bitmap` dan `Image`.
- **System.Windows.Forms:** Diperlukan untuk elemen UI seperti `PictureBox`, `MessageBox`, `EventArgs`, dll.
- **System.Linq:** Menyediakan fitur kueri data (meski di kode ini penggunaannya implisit/sedikit).
- **System.Collections.Generic:** Wajib ada karena kode ini menggunakan `List<T>`.

2. Deklarasi Class dan Variabel (Fields)

C#

```
namespace MiniPhotoshop.Helpers  
{  
    public class DragDropManager
```

```
{
    private readonly PictureBox _mainCanvas;
    private readonly List<PictureBox> _thumbnails;
    private readonly PictureBox _histogram;
    private readonly ToolStripMenuItem _viewMenu;

    private readonly ImageEditorService _editorService;
    private readonly UIManager _toolManager;
```

- **namespace:** Mengelompokkan kelas ini ke dalam folder/kategori Helpers.
- **private readonly ...:** Variabel-variabel ini menyimpan referensi ke komponen UI dan Service. Kata kunci readonly berarti variabel ini hanya bisa diisi sekali (biasanya di Constructor) untuk keamanan memori.
 - `_mainCanvas`: Referensi ke area gambar utama.
 - `_thumbnails`: Daftar (List) kotak gambar kecil.
 - `_editorService`: Penghubung ke logika pemrosesan gambar.
 - `_toolManager`: Penghubung untuk mengatur tombol-tombol UI (aktif/mati).

3. Constructor

C#

```
public DragDropManager(
    PictureBox mainCanvas, List<PictureBox> thumbnails, PictureBox histogram,
    ImageEditorService editorService, UIManager toolManager, ToolStripMenuItem viewMenu)
{
    _mainCanvas = mainCanvas;
    _thumbnails = thumbnails;
    _histogram = histogram;
    _editorService = editorService;
    _toolManager = toolManager;
    _viewMenu = viewMenu;
}
```

- Ini adalah metode yang pertama kali dijalankan saat kelas dibuat (`new DragDropManager(...)`).
- Fungsinya adalah **Dependency Injection**: Menerima objek-objek dari Form utama dan menyimpannya ke dalam variabel lokal (`_mainCanvas`, dll) agar bisa dikendalikan oleh kelas ini.

4. Registrasi Event (RegisterDragDropEvents)

C#

```
public void RegisterDragDropEvents()
{
    _mainCanvas.DragEnter += OnCanvas_DragEnter;
    _mainCanvas.DragDrop += OnCanvas_DragDrop;
    _mainCanvas.MouseDown += OnCanvas_MouseDown;

    foreach (var thumbnail in _thumbnails)
    {
        thumbnail.MouseDown += OnThumbnail_MouseDown;
        thumbnail.DragEnter += OnThumbnail_DragEnter;
        thumbnail.DragDrop += OnThumbnail_DragDrop;
    }
}
```

- Metode ini "menempelkan" (subscribe) fungsi logika ke kejadian (event) mouse.
- **+=**: Operator untuk menambahkan event handler.
- **DragEnter**: Dijalankan saat mouse (yang sedang menyeret sesuatu) *masuk* ke area komponen.
- **DragDrop**: Dijalankan saat mouse *dilepas* di area komponen.
- **MouseDown**: Dijalankan saat tombol mouse *ditekan* (awal mula seret).
- **foreach**: Melakukan perulangan agar *setiap* thumbnail di dalam list memiliki kemampuan drag-and-drop yang sama.

5. Logika Geser Gambar (LoadImageToThumbnail)

C#

```
public void LoadImageToThumbnail(Bitmap image)
{
    // Cek apakah slot terakhir (index 3) sudah terisi. Jika ya, penuh.
    if (_thumbnails[3].Image != null)
```

```

{
    MessageBox.Show(...); // Menampilkan pesan error
    return; // Hentikan fungsi
}

// Algoritma Geser (Shift Right)
for (int i = _thumbnails.Count - 2; i >= 0; i--)
{
    _thumbnails[i + 1].Image = _thumbnails[i].Image;
}

// Masukkan gambar baru di posisi pertama
_thumbnails[0].Image = image;
}

```

- Fungsi ini dipanggil saat user ingin menyimpan hasil edit ke thumbnail.
- **Logika Antrian Geser:**
 1. Cek slot paling kanan (index 3). Jika ada isinya, berarti penuh (karena max 4 slot).
 2. **Loop mundur (i--):** Mulai dari index ke-2 mundur ke 0.
 3. Setiap gambar dipindahkan ke slot di kanannya (i+1 mengambil gambar dari i).
Contoh: Gambar di slot 0 pindah ke slot 1.
 4. Setelah semua digeser ke kanan, slot 0 menjadi kosong (atau tertimpa), lalu diisi dengan gambar baru (image).

6. Event Handler: Memulai Drag (MouseDown)

Untuk Thumbnail:

C#

```

private void OnThumbnail_MouseDown(object sender, MouseEventArgs e)
{
    var clickedThumbnail = (PictureBox)sender; // Ambil thumbnail yg diklik
    if (clickedThumbnail.Image == null || e.Button != MouseButtons.Left) return; // Validasi

    var dragData = new DataObject(); // Buat paket data
    dragData.SetData(DataFormats.Bitmap, clickedThumbnail.Image); // Masukkan gambar ke
paket
    dragData.SetData("SourceThumbnail", clickedThumbnail); // Tandai asalnya dari thumbnail mana

```

```
clickedThumbnail.DoDragDrop(dragData, DragDropEffects.Copy); // Mulai proses drag
}
```

- Kode ini membungkus gambar dan info asal gambar (SourceThumbnail) ke dalam DataObject. Fungsi .DoDragDrop memberitahu Windows bahwa proses seret dimulai.

Untuk Canvas Utama:

C#

```
private void OnCanvas_MouseDown(object sender, MouseEventArgs e)
{
    // Validasi ketat: Harus ada gambar, bukan mode crop (Cursors.Cross), klik kiri.
    if (!_editorService.IsImageLoaded || _mainCanvas.Cursor == Cursors.Cross || e.Button !=
        MouseButton.Left || _mainCanvas.Image == null) return;

    var dragData = new DataObject();
    dragData.SetData(DataFormats.Bitmap, new Bitmap(_mainCanvas.Image)); // Bungkus
gambar
    _mainCanvas.DoDragDrop(dragData, DragDropEffects.Copy); // Mulai drag
}
```

7. Event Handler: Menerima Drag (DragEnter)

C#

```
private void OnCanvas_DragEnter(object sender, DragEventArgs e)
{
    // Cek apakah yang diseret adalah Bitmap?
    if (e.Data.GetDataPresent(DataFormats.Bitmap))
        e.Effect = DragDropEffects.Copy; // Ubah kursor jadi ikon "Copy"
    else
        e.Effect = DragDropEffects.None; // Ubah kursor jadi ikon "Dilarang"
}
```

- Logika ini sama untuk OnThumbnail_DragEnter. Fungsinya hanya visual, memberi tahu user apakah mereka boleh menjatuhkan (drop) objek di sini atau tidak.

8. Event Handler: Melepas Gambar (DragDrop)

Saat Melepas di Canvas Utama (OnCanvas_DragDrop):

C#

```
private void OnCanvas_DragDrop(object sender, DragEventArgs e)
{
    // 1. Ambil gambar dari paket data
    Bitmap droppedImage = (Bitmap)e.Data.GetData(DataFormats.Bitmap);

    // 2. Kirim ke Logic Service untuk diproses
    _editorService.InitialzeImage(droppedImage);

    // 3. Tampilkan gambar ke layar
    _mainCanvas.Image = _editorService.GetRestoredImage();

    // 4. Aktifkan tool UI
    _toolManager.ResetControls();
    _toolManager.EnableTools();
    _viewMenu.Enabled = true;

    // 5. JIKA gambar berasal dari Thumbnail, hapus gambar di thumbnail asal (Pindahkan)
    if (e.Data.GetDataPresent("SourceThumbnail"))
    {
        var sourceThumbnail = (PictureBox)e.Data.GetData("SourceThumbnail");
        sourceThumbnail.Image = null;
    }
}
```

- Ini adalah inti logika memuat gambar untuk diedit. Jika user menarik dari thumbnail ke canvas, gambar di thumbnail dihapus (karena diasumsikan dipindah untuk diedit).

Saat Melepas di Thumbnail (OnThumbnail_DragDrop):

C#

```
private void OnThumbnail_DragDrop(object sender, DragEventArgs e)
{
    var droppedOnThumbnail = (PictureBox)sender; // Thumbnail tujuan
    Bitmap droppedImage = (Bitmap)e.Data.GetData(DataFormats.Bitmap);

    // KASUS A: Geser dari Thumbnail ke Thumbnail lain
    if (e.Data.GetDataPresent("SourceThumbnail"))
    {
        var sourceThumbnail = (PictureBox)e.Data.GetData("SourceThumbnail");
        // Jika tidak menjatuhkan diri sendiri
        if (sourceThumbnail != droppedOnThumbnail)
        {
            sourceThumbnail.Image = null; // Hapus asal
            droppedOnThumbnail.Image = droppedImage; // Isi tujuan
        }
    }

    // KASUS B: Geser dari Canvas ke Thumbnail (Simpan sementara)
    else
    {
        droppedOnThumbnail.Image = droppedImage; // Simpan di thumbnail

        // Reset Canvas (Bersihkan area kerja)
        _mainCanvas.Image = null;
        _histogram.Image = null;
        _editorService.ClearImage();
        _toolManager.DisableTools();
        _viewMenu.Enabled = false;
        _toolManager.ResetControls();
    }
}
```

9. Fungsi Helper (GetLoadedThumbnailCount)

C#

```

public int GetLoadedThumbnailCount()
{
    int count = 0;
    foreach (var tb in _thumbnails)
    {
        if (tb.Image != null) count++; // Hitung jika ada gambarnya
    }
    return count;
}

```

- Fungsi sederhana untuk menghitung berapa banyak thumbnail yang sedang terisi gambar. Berguna untuk logika validasi di form operasi matematika (misalnya operasi AND/OR butuh minimal 2 gambar).

Ringkasan Alur Kerja

1. **Start:** User menekan klik kiri pada gambar (di Canvas atau Thumbnail). Event `MouseDown` membuat paket data (`DataObject`).
2. **Move:** User menggeser mouse. Event `DragEnter` di komponen tujuan mengecek isi paket. Jika isinya `Bitmap`, kursor berubah jadi tanda "Copy".
3. **Drop:** User melepas klik.
 - Jika drop di **Canvas**: Gambar dimuat ke `_editorService`, tools diaktifkan.
 - Jika drop di **Thumbnail**: Gambar disimpan di slot tersebut. Jika asalnya dari Canvas, Canvas dikosongkan (di-reset).